Research Paper Explorer

Rapport de projet — Generative AI (ENSIAS)

Assistant de recherche bibliographique fondé sur l'IA générative : recherche hybride et RAG avec citations vérifiées sur le dataset arXiv de Kaggle, comparaison des papiers cités et rapport PDF.

Version du 25/10/2026

Dépôt : github.com/Sabbarso/agent-ai-

Sommaire

- 1. Objectif et vue d'ensemble
- 2. Les données : le dataset arXiv
- 3. Architecture générale
- 4. Les outils et technologies utilisés
- 5. Le pipeline, étape par étape
- 6. Communication entre les composants
- 7. Garde-fous contre les hallucinations
- 8. L'interface utilisateur
- 9. Tests et qualité
- 10. Limites et pistes d'amélioration
- 11. Commandes pour reproduire le projet
- 12. Glossaire

1. Objectif et vue d'ensemble

Un étudiant ou un chercheur qui commence un sujet se heurte à deux problèmes : le volume (des dizaines de milliers de papiers par an rien qu'en traitement automatique du langage) et la fiabilité des assistants IA, qui inventent facilement des références. **Research Paper Explorer** répond aux deux : il permet de **trouver** les bons papiers, de **poser des questions** dont chaque affirmation est rattachée à un papier réel et vérifié, de **comparer** les papiers cités et d'en **exporter un rapport PDF**.

Étape	Contenu	Résultat
Données	lecture streaming, filtre, nettoyage, enrichissement	papers.parquet (20 000 papiers)
Index	embeddings + ChromaDB, BM25	recherche hybride avec filtres
LLM et RAG	client LLM unique, réponses citées et vérifiées	Chat avec citations [arXiv:ID]
Comparaison	extraction structurée des papiers cités	tableau, synthèse, rapport PDF
Texte intégral	PDF arXiv découpés par section	citations [arXiv:ID §Section]
Interface	Streamlit : Chat et Recherche	historique des conversations enregistré

Le code représente 38 modules Python (environ 4 200 lignes) dans *src/rpe/*, couverts par 113 tests automatisés.

2. Les données : le dataset arXiv

La source unique est le dataset arXiv publié par Cornell University sur Kaggle : un fichier **JSON Lines** de 5,6 Go, un objet JSON par ligne et par papier (identifiant, titre, auteurs, catégories, résumé, commentaires, versions…). Le fichier est trop gros pour être chargé en mémoire : il est **lu ligne par ligne** (streaming) avec la bibliothèque orjson.

Chiffres réels obtenus sur le dump

Étape	Nombre	Commentaire
Lignes lues	3 215 335	0 ligne invalide
Écartées sans décodage JSON	3 092 613	pré-filtre sur les octets « cs.CL » (96 %)
Papiers de la catégorie cs.CL	122 720	au moins une catégorie cs.CL
… publiés depuis le 01/01/2023	81 503	date = versions[0].created
Échantillon conservé	20 000	tirage aléatoire reproductible (seed 42)

Répartition : 2023 : 3 304 · 2024 : 5 063 · 2025 : 5 916 · 2026 : 5 717 papiers. 16,8 % des papiers ont un lien vers du code, 36,3 % sont acceptés dans une conférence ou une revue, et 28,7 % ont une venue identifiée (EMNLP 829, ACL 769, ACL Findings 425, EMNLP Findings 411, ICLR 351, NeurIPS 310…).

Nettoyage et enrichissement

- **Date de publication** : première version (*versions[0].created*, format RFC 2822) et non la date de mise à jour.
- **Auteurs** : reconstruits depuis *authors_parsed*, accents LaTeX (\'e → é) convertis en Unicode.

- **Venue et acceptation** : expressions régulières sur les commentaires et la référence de revue (« Accepted at ACL 2024 », « To appear in NeurIPS'23 », « Findings of EMNLP »…), 27 venues reconnues, formulations négatives (« under review ») détectées.
- **Code** : premier lien github.com ; **pages / figures** : « 8+2 pages, 3 figures ».
- **Échantillonnage par réservoir** : tirage uniforme en une seule passe, avec une mémoire bornée à 20 000 papiers.

3. Architecture générale

Le schéma montre les blocs et le sens des données. En bleu les données brutes, en jaune les fichiers stockés sur disque, en vert les traitements, en orange ce qui fait appel au LLM et en violet l'interface.

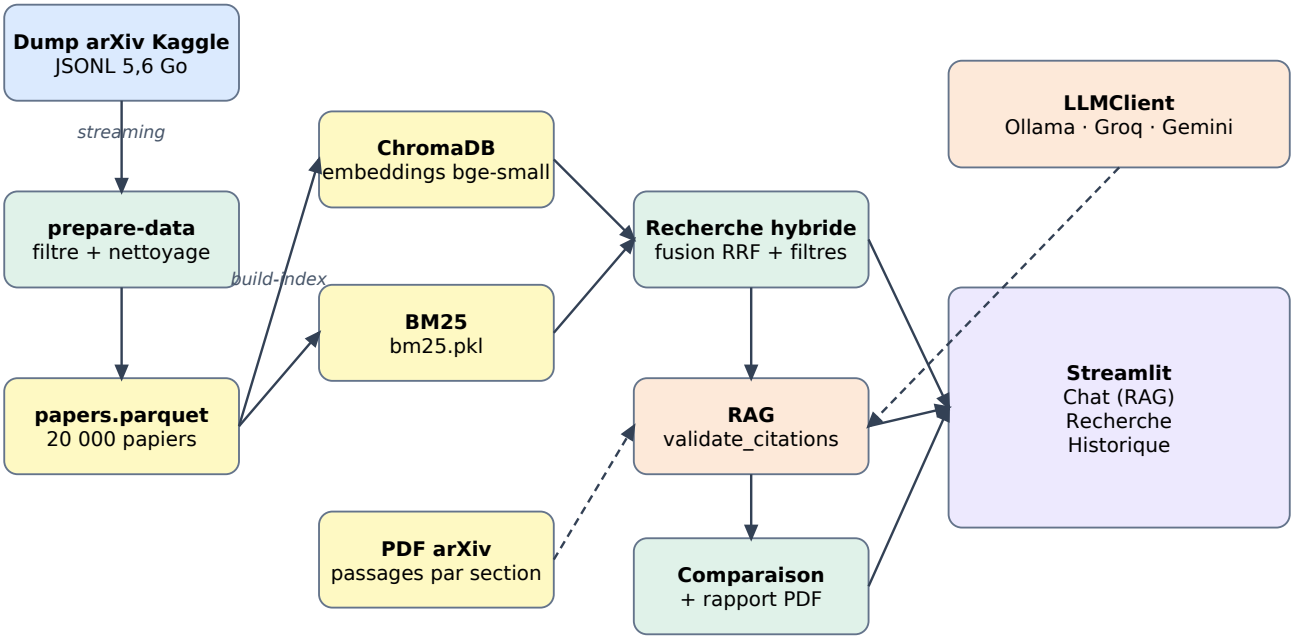


Figure 1 — Architecture : des données brutes à l'interface.

Chaque étape produit des **fichiers** que l'étape suivante relit : chacune est indépendante, reprenable après une interruption et testable séparément. Un mode **--dev** travaille sur 1 000 papiers avec des fichiers séparés, pour itérer vite sans écraser le corpus complet.

Organisation du code

Dossier	Rôle
src/rpe/config.py	lecture de config.yaml (paramètres) et .env (secrets)
src/rpe/data/	streaming, nettoyage, enrichissement, accès aux papiers
src/rpe/index/	embeddings, ChromaDB, BM25, filtres, recherche hybride
src/rpe/llm/	client LLM, prompts centralisés, schémas Pydantic
src/rpe/rag/	RAG, vérification des citations, comparaison des papiers
src/rpe/pdf/	téléchargement des PDF, découpage en sections, passages
src/rpe/reports/	génération des rapports PDF
src/rpe/ui/, app/	interface Streamlit et historique des conversations
src/rpe/resources.py	chargement partagé des index et du LLM
src/rpe/cli.py	ligne de commande (Typer)

4. Les outils et technologies utilisés

Chaque outil a un rôle précis ; le tableau indique pourquoi il est là et où.

Outil	Rôle dans le projet	Où
Python 3.12	langage principal (typé, testé)	partout
orjson	décodage JSON très rapide, ligne par ligne, du dump de 5,6 Go	data/load.py
pandas + pyarrow	tableaux de papiers, stockage Parquet compressé	data/
sentence-transformers	modèle BAAI/bge-small-en-v1.5 : chaque « titre. résumé » devient un vecteur de 384 nombres	index/embed.py
ChromaDB	base vectorielle persistante : vecteurs les plus proches, filtres sur l'année, la venue, le code…	index/vector_store.py
rank-bm25	recherche lexicale classique (mots exacts, pondération BM25)	index/bm25.py
openai (client)	protocole standard pour parler à tout LLM compatible : Ollama local, Groq, Gemini, OpenAI	llm/client.py
Ollama + qwen2.5:3b	LLM exécuté localement, sans clé API ni coût	.env
Pydantic	schémas des sorties du LLM : JSON validé, sinon corrigé par le LLM	llm/schemas.py
PyMuPDF	extraction du texte des PDF arXiv	pdf/parse.py
httpx	téléchargement des PDF, avec pause entre les requêtes	pdf/download.py
ReportLab	rapports PDF (comparaison, ce document)	reports/
Streamlit	application web : Chat et Recherche	app/
Typer	ligne de commande python -m rpe.cli …	cli.py
pytest + ruff	tests automatisés et contrôle de qualité du code	tests/
Git / GitHub	versionnement et partage du code	dépôt

5. Le pipeline, étape par étape

Préparation des données

Lecture en streaming, pré-filtre sur les octets (une ligne qui ne contient pas « cs.CL » est ignorée sans être décodée), filtre exact sur les catégories et les dates, nettoyage, enrichissement par expressions régulières, échantillonnage par réservoir et écriture atomique du Parquet. Une passe sur le dump prend environ 1 min 15 s.

Index et recherche hybride

Deux moteurs complémentaires : **BM25** retrouve les mots exacts (noms de datasets, acronymes) et la recherche **dense** retrouve le sens (« reduce hallucination » ≈ « improve factuality »). Chacun propose 100 candidats ; la **Reciprocal Rank Fusion** les combine : score = Σ 1/(60 + rang). Les embeddings sont calculés par parties de 2 048 papiers pour pouvoir reprendre après une interruption.

Couche LLM et RAG

Le **LLMClient** encapsule tous les appels (section 6). Le **RAG** récupère les 5 papiers les plus pertinents, les numérote dans le prompt avec leur identifiant, demande une réponse citée, puis vérifie chaque citation.

Comparaison des papiers cités et rapport PDF

Les papiers cités dans la réponse (5 au maximum) sont décrits en un seul appel LLM groupé (problème, méthode, datasets, métriques, résultats, limites), mis en cache par papier. Une synthèse comparative citée est rédigée. Le rapport PDF reprend la réponse, une fiche par papier, un tableau comparatif en paysage et les références.

Texte intégral (PDF)

Jusqu'à 300 PDF téléchargés depuis arxiv.org, avec 3 s de pause entre deux requêtes. Le texte est découpé en sections (Introduction, Method, Experiments…) puis en passages de 1 500 caractères avec chevauchement, indexés dans ChromaDB. Le RAG sur un papier cite alors la section ([arXiv:ID §Method]) et la comparaison utilise le texte complet.

Interface

Application Streamlit qui s'ouvre sur le Chat, avec la Recherche en seconde page et l'historique des conversations enregistré (section 8).

6. Communication entre les composants

Les composants ne s'appellent pas en réseau : ce sont des modules Python qui échangent des **objets typés** (Pydantic) en mémoire et des **fichiers** sur disque. Seuls deux échanges sortent de la machine : les requêtes vers le LLM (en local avec Ollama sur http://localhost:11434) et le téléchargement des PDF sur arxiv.org.

6.1 Les fichiers échangés

Fichier	Produit par	Lu par	Format
papers.parquet	prepare-data	index, RAG, interface	Parquet
embeddings.npy + ids.json	build-index	build-index (reprise)	NumPy
chroma/	build-index, index-pdfs	recherche dense, RAG	base vectorielle
bm25.pkl	build-index	recherche lexicale	objet sérialisé
data/pdfs/	fetch-pdfs	index-pdfs, comparaison	PDF + JSON
data/cache/llm/	LLMClient	LLMClient	JSON par empreinte
data/cache/compare/	comparaison	comparaison	JSON par papier
data/conversations/	page Chat	page Chat	JSON par conversation

6.2 Le LLMClient : un point d'entrée unique vers le modèle

Tous les modules passent par la même classe. Elle vérifie d'abord le cache disque (clé = empreinte sha256 du modèle et des messages) : une question déjà posée ne coûte aucun appel. Sinon elle appelle l'endpoint compatible OpenAI avec un délai maximal, relance avec une attente croissante en cas d'erreur réseau, puis valide le JSON reçu contre le schéma Pydantic attendu. Si le JSON est invalide, le LLM reçoit sa propre réponse et l'erreur, et doit la corriger.

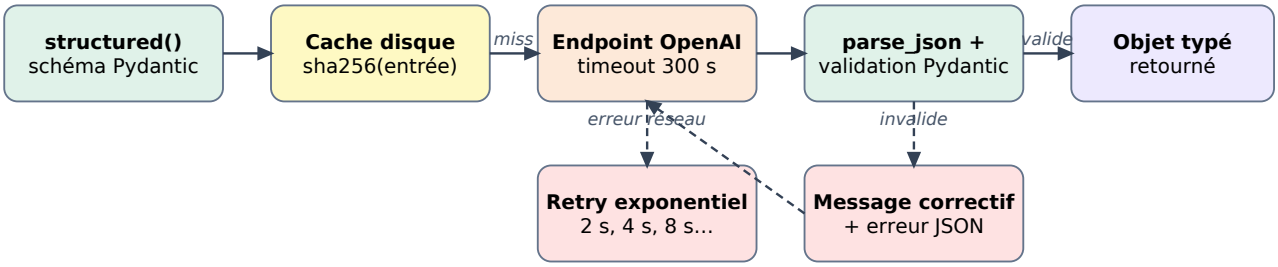


Figure 2 — Cycle d'un appel structuré au LLM.

```
# Exemple : la comparaison demande un objet typé, pas du texte libre
lot = llm.structured(prompt, ComparisonBatch, system=COMPARE_EXTRACT_SYSTEM)
lot.papers[0].datasets   # -> ["SQuAD", "Natural Questions"]  (liste validée)
llm.stats                # -> LLM calls=1 cache_hits=0 retries=0 parse_failures=0
```

6.3 Le trajet d'une question dans le Chat

Voici ce qui se passe, étape par étape, quand l'utilisateur pose une question :

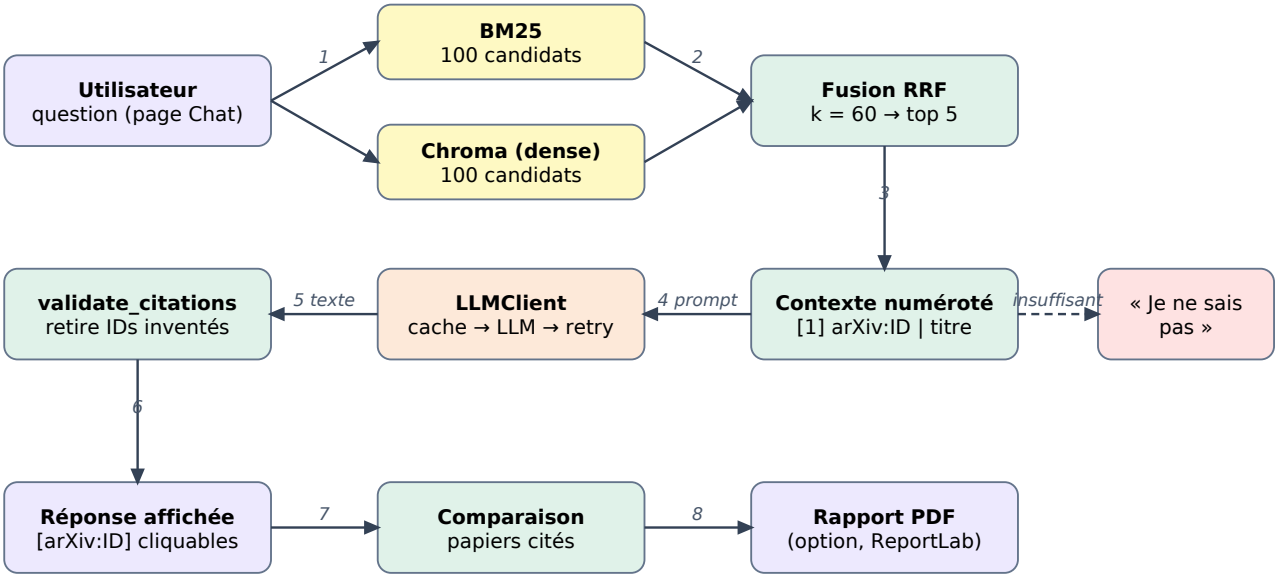


Figure 3 — Séquence d'une question RAG (numéros = ordre des échanges).

- **1–2.** La question part vers les deux moteurs ; BM25 et ChromaDB renvoient chacun une liste classée d'identifiants arXiv, fusionnée par RRF.
- **3.** Les 5 meilleurs papiers forment un contexte numéroté : « [1] arXiv:2503.10677 | titre (année, venue) » suivi du résumé (et des passages du PDF s'il a été indexé).
- **4–5.** Le prompt impose de citer chaque affirmation avec un identifiant du contexte, ou de répondre INSUFFICIENT_CONTEXT ; le LLM renvoie un texte.
- **6.** *validate_citations()* retire tout identifiant absent du contexte (et toute section inventée). S'il ne reste aucune citation valide, la réponse devient « Je ne sais pas ».
- **7–8.** Les papiers cités sont comparés (un appel groupé, mis en cache) ; le rapport PDF réutilise cette comparaison, sans nouvel appel au LLM.

6.4 L'interface et le reste du code

Les pages Streamlit ne contiennent pas de logique métier : elles appellent les mêmes fonctions que la ligne de commande, via l'objet *Resources* qui charge à la demande (et une seule fois, grâce à *st.cache_resource*) les papiers, les index et le client LLM. Chaque conversation est enregistrée dans un fichier JSON (questions, réponses, comparaisons) et rechargée quand on la sélectionne. La page Recherche communique avec le Chat en déposant dans la session le papier choisi ou la question, puis en changeant de page (*st.switch_page*).

7. Garde-fous contre les hallucinations

Un LLM peut inventer une référence plausible. Le projet ne fait jamais confiance au texte généré sans vérification :

Risque	Garde-fou
Citation d'un papier inexistant ou hors contexte	validate_citations() retire tout [arXiv:ID] absent du contexte
Section inventée ([arXiv:ID §3.3])	la section est supprimée si elle ne figure pas dans le contexte
Réponse sans source	aucune citation valide → « Je ne sais pas »
Résultats inventés (comparaison)	un résultat contenant un chiffre absent du texte source est écarté
Information absente	« Non mentionné dans le résumé » au lieu d'une valeur devinée
Références du rapport PDF	générées par le code à partir des seuls papiers cités et validés
JSON mal formé	validation Pydantic + nouvel essai avec le message d'erreur

8. L'interface utilisateur

L'application (*streamlit run app/streamlit_app.py*, puis http://localhost:8501) s'ouvre directement sur le Chat et comporte deux pages :

- **Chat (RAG)** : réponse avec citations cliquables et « Contexte récupéré » ; dessous, la **comparaison des papiers cités** (tableau, onglets Méthodes / Datasets & métriques / Résultats / Limitations, synthèse, export Markdown / CSV) et l'option **Générer le rapport PDF**. Le menu *Options* règle la portée (corpus ou un papier), la langue et l'affichage de la comparaison.
- **Historique** : dans la barre latérale, « Nouvelle conversation », la liste des conversations enregistrées (un clic rouvre la conversation), le téléchargement en Markdown et la suppression.
- **Recherche** : requête, mode (hybride, BM25, dense), filtres (années, catégories, venue, code, accepté). Chaque résultat propose « Interroger avec le RAG » (ouvre le Chat sur ce papier) et un TL;DR.

Aucun réglage technique n'est affiché : l'application choisit seule le corpus disponible (complet, sinon échantillon).

9. Tests et qualité

113 tests automatisés passent (pytest) et le code respecte les règles de ruff. Les tests n'appellent ni le réseau ni un vrai LLM : un faux LLM scripté renvoie des réponses prévues, ce qui rend les tests rapides (environ 20 s) et reproductibles.

Fichier de tests	Tests	Ce qui est vérifié
test_enrich_comments	36	venues, années, acceptation, GitHub, pages / figures
test_llm_rag	16	citations, cache, retries, JSON invalide, « Je ne sais pas »
test_clean / test_index	15 / 15	dates, catégories, RRF, filtres, déterminisme
test_compare / test_report	8 / 8	papiers cités, « Non mentionné », PDF valide
test_pdf / test_ui / test_conversations	6 / 5 / 4	sections, rendu des pages, historique

10. Limites et pistes d'amélioration

Limites

- Un LLM local de 3 milliards de paramètres sur CPU est lent (1 à 4 minutes par réponse) et moins précis qu'un modèle hébergé. Un fournisseur hébergé (Groq, Gemini) l'accélère fortement, sans changer le code (seulement le fichier .env).
- Les venues hors liste (FAccT, WMT…) ne sont pas reconnues.
- Le découpage des PDF en sections repose sur des titres usuels.
- Le corpus est limité à la catégorie cs.CL depuis 2023 (paramétrable dans config.yaml).

Pistes d'amélioration

- Re-classement des résultats par un cross-encoder après la fusion RRF.
- Évaluation chiffrée (Recall@k, MRR, fidélité) sur un jeu annoté à la main.
- Analyse du domaine : thèmes et tendances, graphe de connaissances, agent.
- Mise à jour incrémentale du corpus avec les nouveaux papiers de l'API arXiv.

11. Commandes pour reproduire le projet

```
py -3.12 -m venv .venv ; .venv\Scripts\activate
pip install -r requirements.txt ; pip install -e . --no-deps
copy .env.example .env          # LLM_BASE_URL, LLM_MODEL (ex. Ollama local)

python -m rpe.cli prepare-data          # 1. corpus     (--dev : 1 000 papiers)
python -m rpe.cli build-index           # 2. index
python -m rpe.cli fetch-pdfs            # 3. optionnel : PDF arXiv
python -m rpe.cli index-pdfs
streamlit run app/streamlit_app.py      # 4. application : http://localhost:8501

python -m rpe.cli search "retrieval-augmented generation" --year-min 2024
python -m rpe.cli ask "How can retrieval reduce hallucinations?" --lang fr --pdf rapport.pdf

pytest ; ruff check .                   # tests et qualité
```

12. Glossaire

Terme	Définition
Embedding	vecteur de nombres qui représente le sens d'un texte ; deux textes proches ont des vecteurs proches
BM25	score classique de recherche par mots-clés, qui favorise les mots rares et présents
RRF	Reciprocal Rank Fusion : combine plusieurs classements, score = Σ 1/(k + rang)
RAG	Retrieval-Augmented Generation : le LLM répond à partir de documents récupérés et non de sa mémoire
Pydantic	bibliothèque de validation de données typées en Python
Ollama	logiciel qui exécute des LLM en local et expose une API compatible OpenAI
Streaming	lecture d'un fichier ligne par ligne, sans le charger entièrement en mémoire